SIMATS ENGINEERING

PROGRAMMING IN JAVA

ASSIGNMENT 4 – CO4

EVENT-DRIVEN GUI DESIGN

Course: Programming in Java (CSA09)

Course Outcome: CO4 – Applets, the event delegation model, listeners and adapter classes, inner and anonymous classes, AWT components and controls, layout managers, menus and dialogs, Swing components and GUI–JDBC integration (Sessions 49–56)

Nature of Assignment: Individual

Duration: One Week

Total Marks: 50

Submission Date: _____

Bloom's Levels Assessed: K4 – Analyse and K5 – Evaluate (the level of each task is stated under the task heading)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA09-<RegisterNumber>-Assignment4** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the Java source code, input files and output screenshots.
- Include a README.md with your name, register number, Java version, the steps to compile and run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/Pavithra16780/CSA09-192472155-Assignment4.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

PURPOSE OF THE ASSIGNMENT

This assignment evaluates your ability to **design an event-driven user interface before building it**, and to keep the interface separate from the application logic.

You are expected to demonstrate the following progression:

> **Interpret the Requirement → Identify Components and Events → Design the Layout → Design the Handlers → Implement → Trace → Challenge → Refine**

Where pseudocode is required, it must follow the SIMATS Engineering – Standard for Algorithmic Pseudocode. Implementation must use AWT and/or Swing.

TASK 1 – GUI DESIGN REVIEW

Bloom's Level: K4 – Analyse

Diagnose and Repair a Poor Event-Handling Design

6 Marks

1.1 Introduction

A Graphical User Interface (GUI) allows users to interact with a computer application through visual components such as buttons, text fields, labels, menus, and dialog boxes.

In Java, event handling is used to respond to user actions. When a user clicks a button or enters information, an event is generated. A listener receives the event and performs the required operation.

A good GUI design should separate the user interface from the application logic. This makes the program easier to test, understand, maintain, and modify.

1.2 Problem Statement

A junior developer has developed a student fee-payment window. The program uses one listener for all buttons and performs the complete payment logic inside the event-handling method.

This design creates several problems, including poor separation of responsibilities, input-validation issues, database security risks, and an unresponsive window.

The objective is to identify the design problems and redesign the application using a separate FeeService class.

1.3 Task 1.1 – Diagnose the Design

The given program has the following design deficiencies:

S.No.	Design Deficiency	Explanation and User Impact
1	Poor separation of interface and logic	Database operations and payment calculations are written inside the GUI listener. This makes the program difficult to maintain and test.
2	Incorrect string comparison	The program uses == to compare the action command. String content should be compared using equals().

3	No input validation	Empty registration numbers, negative amounts, and invalid numeric values may cause errors.
4	Database work on the event-dispatch thread	Database operations can take time, causing the GUI to freeze while processing a payment.
5	Use of Thread.sleep()	Sleeping for three seconds blocks the event-dispatch thread and prevents the window from responding normally.
6	Poor user feedback	Printing "Paid" only to the console does not clearly inform the user through the GUI.
7	Receipt button is not implemented	The program treats every non-Pay event as a Clear action. Therefore, clicking Receipt may clear the fields instead of generating a receipt.
8	SQL injection risk	The SQL statement directly joins user input into the query. Malicious input may change the query.
9	Database connection not closed	The database connection and statement are not properly released, potentially causing resource leaks.
10	Window closing is not handled	The program does not define suitable window-closing behaviour.
11	Exceptions are not handled	Database operations can throw checked exceptions, but the program does not handle them properly.

Explanation of Important Deficiencies

1. Separation of Interface and Logic

The GUI should collect input and display output. Payment validation and database operations should be performed by a separate service class.

2. String Comparison

Incorrect:

if (e.getActionCommand() == "Pay")

Correct:

if (e.getActionCommand().equals("Pay"))

The equals() method compares the contents of two strings.

3. Input Validation

The application should check whether the registration number is empty and whether the amount is a valid positive number before performing the payment.

4. Event-Dispatch Thread

Swing and AWT event handling should not be blocked by long-running database operations. A slow database operation can make the window appear frozen.

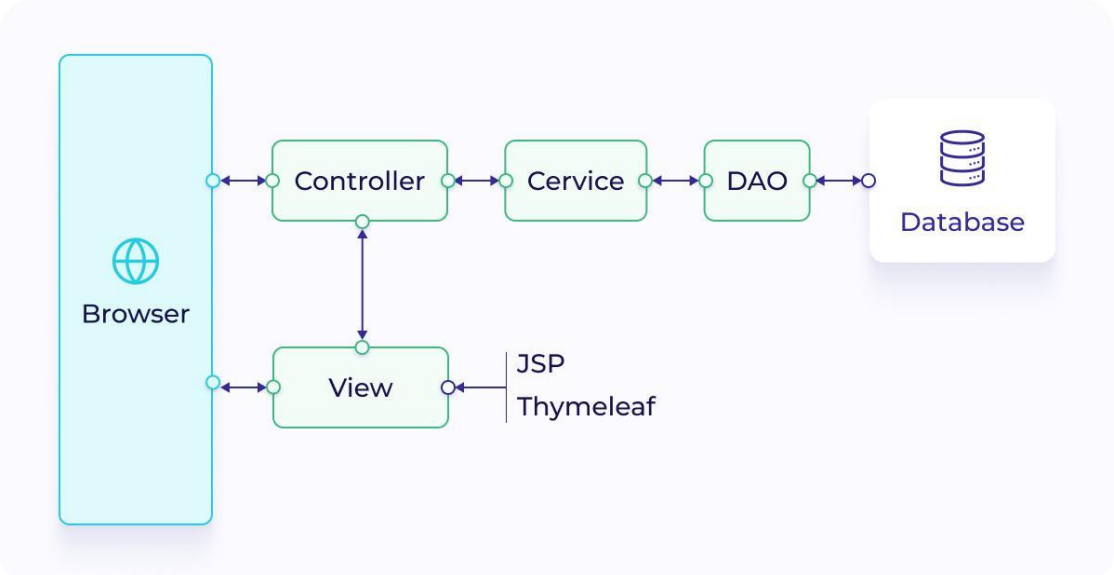
5. SQL Security

PreparedStatement should be used instead of directly joining user input into an SQL query.

1.4 Task 1.2 – Repair the Design

The redesigned application separates the GUI and payment logic.

Proposed Class Structure



FeeWindow Class

- Displays the fee-payment interface.
- Accepts the registration number.
- Accepts the payment amount.
- Provides Pay, Clear, and Receipt buttons.
- Displays success and error messages.
- Sends requests to FeeService.

FeeService Class

- Validates the registration number.
- Checks the payment amount.
- Performs the payment operation.
- Communicates with the database.
- Returns the payment result.

Database Layer

- Establishes the database connection.
- Executes parameterized SQL statements.
- Stores payment information.
- Closes database resources.

FeeService.java

```
import java.sql.*;


public class FeeService {


    private static final String URL =

        "jdbc:mysql://localhost:3306/college";
```

```
private static final String USER = "root";
private static final String PWD = "password";

public String payFee(String regNo, double amount)
        throws Exception {

    if (regNo == null || regNo.trim().isEmpty()) {
        throw new IllegalArgumentException(
            "Registration number cannot be empty"
        );
    }

    if (amount <= 0) {
        throw new IllegalArgumentException(
            "Amount must be greater than zero"
        );
    }

    String sql =
        "INSERT INTO fees (reg_no, amount) VALUES (?, ?)";

    try (
        Connection con =
```

```
            DriverManager.getConnection(URL, USER, PWD);


            PreparedStatement ps = con.prepareStatement(sql)
        ) {
            ps.setString(1, regNo);
            ps.setDouble(2, amount);


            ps.executeUpdate();


            return "Payment successful";
        }
    }
}
```

FeeWindow.java

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;


public class FeeWindow extends JFrame {


    JTextField regNo = new JTextField(15);
    JTextField amount = new JTextField(15);


    JButton pay = new JButton("Pay");
```

```
JButton clear = new JButton("Clear");
JButton receipt = new JButton("Receipt");

JLabel status = new JLabel("Enter payment details");

FeeService service = new FeeService();

public FeeWindow() {

    setTitle("Student Fee Payment");
    setLayout(new GridLayout(5, 2, 5, 5));

    add(new JLabel("Registration Number"));
    add(regNo);

    add(new JLabel("Amount"));
    add(amount);

    add(pay);
    add(clear);

    add(receipt);
    add(status);
```

```
pay.addActionListener(e -> makePayment());

clear.addActionListener(e -> {
    regNo.setText("");
    amount.setText("");
    status.setText("Fields cleared");
});

receipt.addActionListener(e -> {
    JOptionPane.showMessageDialog(
        this, "Receipt feature selected"
    );
});

addWindowListener(new WindowAdapter() {
    @Override
    public void windowClosing(WindowEvent e) {
        dispose();
    }
});

setSize(450, 250);
setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
setVisible(true);
```

```
}

private void makePayment() {

    try {
        String id = regNo.getText();
        double a = Double.parseDouble(amount.getText());

        String message = service.payFee(id, a);

        status.setText(message);

        JOptionPane.showMessageDialog(this, message);

    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(
            this, "Enter a valid amount"
        );

    } catch (IllegalArgumentException e) {
        JOptionPane.showMessageDialog(
            this, e.getMessage()
        );
```

```
        } catch (Exception e) {
            JOptionPane.showMessageDialog(
                this, "Payment failed: " + e.getMessage()
            );
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(FeeWindow::new);
    }
}
```

Implementation note: The example separates the payment service from the window, but the database call is still synchronous in the event handler. For a production application, run the payment operation in a SwingWorker so that the event-dispatch thread remains responsive.

1.5 Task 1.3 – Design Defence

The redesigned program is easier to test and maintain because the GUI and payment logic are separated. The FeeWindow class is responsible for collecting input and displaying results, while FeeService validates the data and performs the payment operation. Changes to the database logic do not require major changes to the GUI. The service class can also be tested independently using different registration numbers and payment amounts. This improves code reuse, readability, debugging, and maintainability.

1.6 Alternative Concept – Separate Listener Objects

Instead of using one listener with an if-else chain, each button can have a separate listener object or Swing Action.

Example:

```
pay.addActionListener(e -> makePayment());


clear.addActionListener(e -> {
    regNo.setText("");
    amount.setText("");
});


receipt.addActionListener(e -> showReceipt());
```

Complexity Analysis

Let k be the number of buttons.

Approach	**Event Dispatch Time**	**Space Complexity**
One listener with if-else	Up to O(k) comparisons	O(1) listener object
Separate listener objects	O(1) direct dispatch per registered listener	O(k) listener objects
Shared Swing Action	Direct action dispatch	Depends on the number of shared Action objects

The one-listener approach may require checking multiple conditions before identifying the action. Separate listeners avoid this comparison chain and make each action easier to manage.

1.7 Conclusion

The original fee-payment design contains several problems related to validation, database access, security, and event handling. The redesigned application separates the GUI from the payment service. Separate listeners and PreparedStatement improve maintainability and security. Long-running payment operations should be performed outside the event-dispatch thread.

TASK 2 – FROM REQUIREMENT TO EVENT-DRIVEN DESIGN

Bloom's Level: K4 – Analyse

Hospital Outpatient Appointment Desk

8 Marks

The outpatient desk needs a window in which the receptionist enters a patient ID, selects a department from a list, chooses a doctor (the doctors shown depend on the department), picks a time slot and books the appointment. The window must show a confirmation or an error, and must ask for confirmation before closing if a booking is in progress.

2.1 Introduction

An event-driven application responds to events generated by user actions. In Java, components such as buttons, lists, text fields, and windows generate events. Listener interfaces are used to handle those events.

An outpatient appointment desk needs an event-driven interface that allows a receptionist to select a patient, department, doctor, and time slot and book an appointment.

2.2 Problem Statement

The hospital outpatient desk requires a GUI application for appointment booking. The receptionist must enter a patient ID, select a department, choose a doctor based on the selected department, select a time slot, and book the appointment.

The system must display confirmation or error messages and ask for confirmation before closing the window when a booking is in progress.

2.3 Objectives

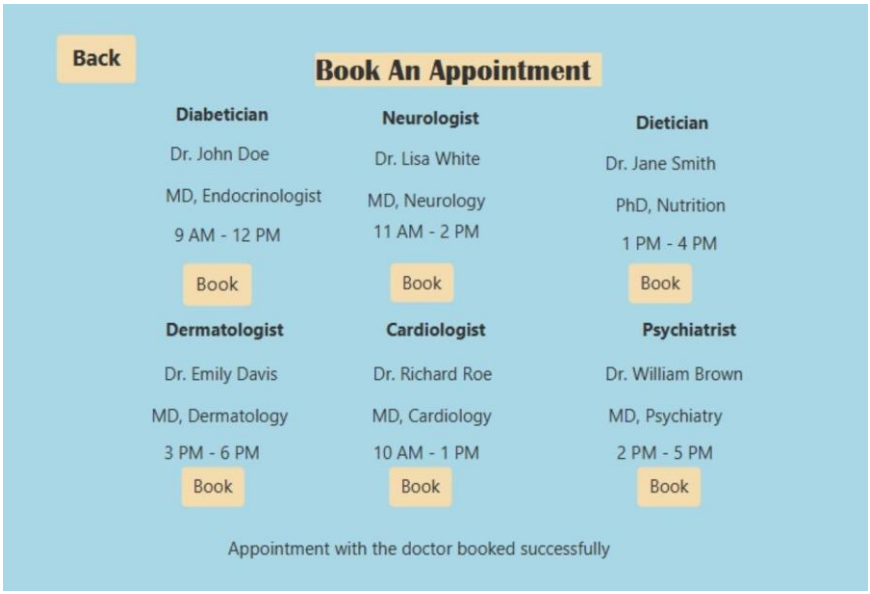
1. To identify event sources and listener interfaces.
2. To design a suitable component hierarchy.
3. To create event-handler pseudocode.
4. To understand window events.
5. To compare database-based doctor loading with preloaded doctor data.

2.4 Task 2.1 – Model the Events

Event Source	Event Object	Listener Interface	Action
Patient ID text field	ActionEvent	ActionListener	Accept or validate patient ID
Department selection	ItemEvent	ItemListener	Refresh the doctor list
Doctor selection	ItemEvent	ItemListener	Update selected doctor
Time slot selection	ItemEvent	ItemListener	Update selected time
Book button	ActionEvent	ActionListener	Validate and book appointment
Cancel button	ActionEvent	ActionListener	Clear or cancel selection
Window closing	WindowEvent	WindowListener / WindowAdapter	Ask for confirmation before closing

2.5 Task 2.2 – Component Hierarchy

The appointment booking interface can be designed using a JFrame with multiple panels.



Component Tree

JFrame – Appointment Desk

├── JPanel – Buttons

│ ├── JButton – Book Appointment

│ └── JButton – Cancel

│

└── JPanel – Status

└── JLabel – Confirmation / Error

Components Used

Component	**Purpose**
JFrame	Main application window
JPanel	Groups related components
JLabel	Displays text labels
JTextField	Accepts patient ID
JComboBox	Selects department, doctor, and time slot
JButton	Performs booking and cancellation
JOptionPane	Displays confirmation and error dialogs

2.6 Task 2.3 – Design the Handler Logic

(a) Pseudocode for Refreshing the Doctor List

ALGORITHM: REFRESH_DOCTOR_LIST

BEGIN

READ selectedDepartment

CLEAR doctorComboBox

```
    IF selectedDepartment is not empty THEN

        GET doctorList from doctorMap
            using selectedDepartment

        FOR EACH doctor IN doctorList

            ADD doctor to doctorComboBox

        END FOR

    END IF

    UPDATE doctorComboBox

END
```

Explanation

1. The receptionist selects a department.
2. The program reads the selected department.
3. The existing doctor list is cleared.
4. The program retrieves the doctors belonging to that department.
5. The doctor names are added to the doctor selection box.
6. The updated list is displayed.

(b) Pseudocode for Validating and Booking an Appointment

```
ALGORITHM: BOOK_APPOINTMENT

BEGIN

  READ patientID
  READ selectedDepartment
  READ selectedDoctor
  READ selectedTimeSlot

  IF patientID is empty THEN

    DISPLAY "Enter patient ID"

  ELSE IF selectedDepartment is empty THEN

    DISPLAY "Select department"

  ELSE IF selectedDoctor is empty THEN

    DISPLAY "Select doctor"

  ELSE IF selectedTimeSlot is empty THEN

    DISPLAY "Select time slot"
```

```
    ELSE

        VALIDATE patientID

        CHECK doctor availability

        IF doctor is available THEN

            SAVE appointment

            DISPLAY "Appointment booked successfully"

        ELSE

            DISPLAY "Selected time slot is unavailable"

        END IF

    END IF

END
```

2.7 Task 2.4 – Trace the Event Sequence

The following sequence describes the execution of the appointment application.

1. The application starts.
2. The main method creates the appointment window.

3. The JFrame is initialized.
4. The components are created and added to panels.
5. Event listeners are registered.
6. The window is displayed.
7. The receptionist enters the patient ID.
8. The receptionist selects a department.
9. An ItemEvent is generated.
10. The ItemListener updates the doctor list.
11. The receptionist selects a doctor and time slot.
12. The Book button is clicked.
13. An ActionEvent is generated.
14. The booking handler validates the input.
15. The appointment is saved.
16. A confirmation message is displayed.
17. When the receptionist attempts to close the window, a WindowEvent is generated.
18. The program checks whether a booking is in progress.
19. A confirmation dialog is displayed if necessary.
20. The window is closed after confirmation.

Window Event Methods

Method	**Purpose**
windowOpened()	Called when the window is opened
windowClosing()	Called when the user attempts to close the window
windowClosed()	Called after the window is closed

windowIconified()	Called when the window is minimized
windowDeiconified()	Called when the window is restored
windowActivated()	Called when the window becomes active
windowDeactivated()	Called when the window becomes inactive

Comparison with Applet Life Cycle

Applet Method	**Purpose**
init()	Initializes the applet
start()	Starts or resumes applet execution
paint()	Displays applet content
stop()	Stops applet activity
destroy()	Releases applet resources

An applet uses predefined life-cycle methods controlled by its hosting environment. A JFrame application is normally created through the main method and responds to window and component events.

2.8 Alternative Concept – Fetch Doctors from Database

Instead of loading doctor information once into a Map<String, List<Doctor>>, the program can fetch the doctor list from the database every time the department changes.

Database-Based Approach

```
String sql =
    "SELECT doctor_name FROM doctors WHERE department = ?";


PreparedStatement ps = con.prepareStatement(sql);
ps.setString(1, selectedDepartment);


ResultSet rs = ps.executeQuery();
```

```
doctorComboBox.removeAllItems();


while (rs.next()) {
   doctorComboBox.addItem(rs.getString("doctor_name"));
}
```

Complexity Analysis

Let:

- d = number of departments.
- m = average number of doctors per department.

Approach	**Department Change Time**	**Additional Memory**
Preloaded Map	O(m) to retrieve and display the selected list	O(dm) for storing doctor lists
Database Fetch	Depends on database query and returned records; approximately O(m) result processing plus query and network overhead	O(m) for the current result list, excluding database-side storage

The Map approach provides quick access but uses memory to store doctor information. Database fetching avoids storing all doctor lists in application memory, but each department change requires a database operation.

2.9 Conclusion

The hospital appointment desk uses event-driven programming to respond to patient and receptionist actions. Listener interfaces handle department changes, doctor selection, booking, and window closing. A component hierarchy makes the interface organized, while validation ensures correct appointment details. Preloading doctor data improves access speed, whereas database fetching keeps the data current.

TASK 3 – FROM SKETCH TO LAYOUT

Bloom's Level: K4 – Analyse

Campus Library Search Screen

9 Marks

The library search screen has: a title bar at the top; a search row with a text field, a category Choice and a Search button; a results area in the centre; a row of Prev, Next and Reserve buttons at the bottom; and a status message. When the window is resized, the results area must grow while the search row and buttons keep their size.

3.1 Introduction

Layout managers are used in Java AWT and Swing to arrange GUI components inside containers. They automatically position and resize components when the application window changes size.

The campus library search screen requires a layout in which the results area grows when the window is resized, while the search row and bottom buttons retain their normal size.

3.2 Problem Statement

A campus library search application must contain:

- A title bar at the top.
- A search row with a text field, category selection, and Search button.
- A results area in the centre.
- Prev, Next, and Reserve buttons at the bottom.
- A status message.

The results area must expand when the window is resized. The search row and bottom controls should remain properly arranged.

3.3 Objectives

1. To understand Java layout managers.
2. To select suitable layouts for each container.

3. To design nested panels.
4. To implement the library search screen.
5. To test resizing behaviour.

3.4 Task 3.1 – Choose the Layout Managers

The following layout managers can be used:

Layout Manager	Application	Reason
BorderLayout	Main JFrame	Separates the top, centre, and bottom regions.
FlowLayout	Search panel	Places the search field, category, and button in a row.
BorderLayout	Results panel	Allows the results area to occupy the available centre space.
FlowLayout	Navigation panel	Arranges Prev, Next, and Reserve buttons in a row.
BorderLayout	Bottom panel	Separates navigation buttons and status message.

Explanation

BorderLayout

BorderLayout divides a container into five regions:

- NORTH
- SOUTH
- EAST
- WEST
- CENTER

The centre region receives the extra available space when the window is resized. Therefore, it is suitable for the library results area.

FlowLayout

FlowLayout arranges components from left to right. It is suitable for the search row and navigation buttons.

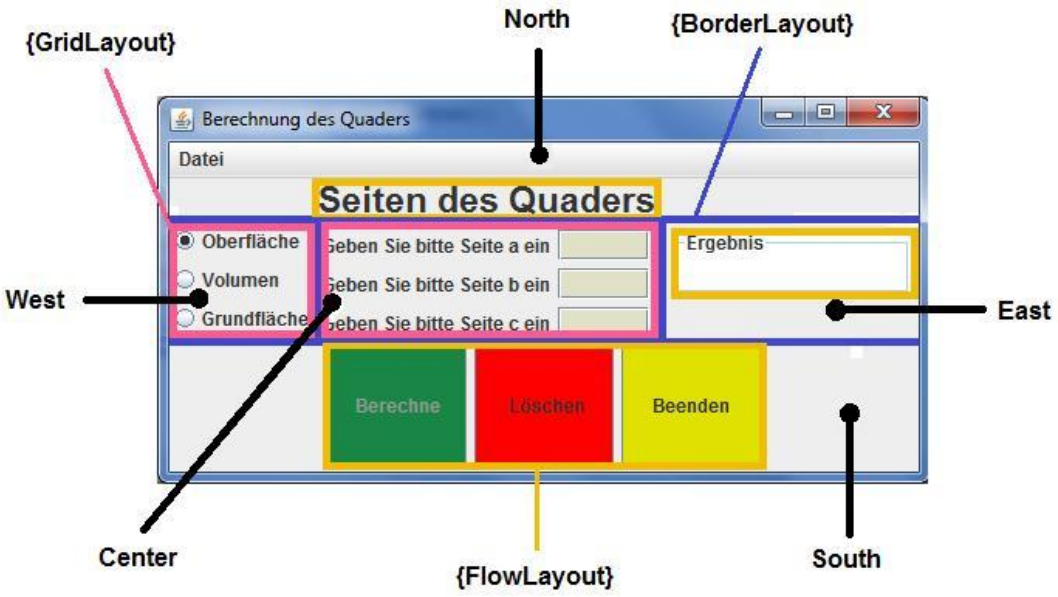
GridLayout

GridLayout divides a container into equal-sized rows and columns. It is useful when components need equal-sized cells, but it is not the best choice for the flexible results area.

CardLayout

CardLayout displays one component at a time. It is useful for switching between different screens, but it is not necessary for the basic library search layout.

3.5 Task 3.2 – Design the Nested Containers



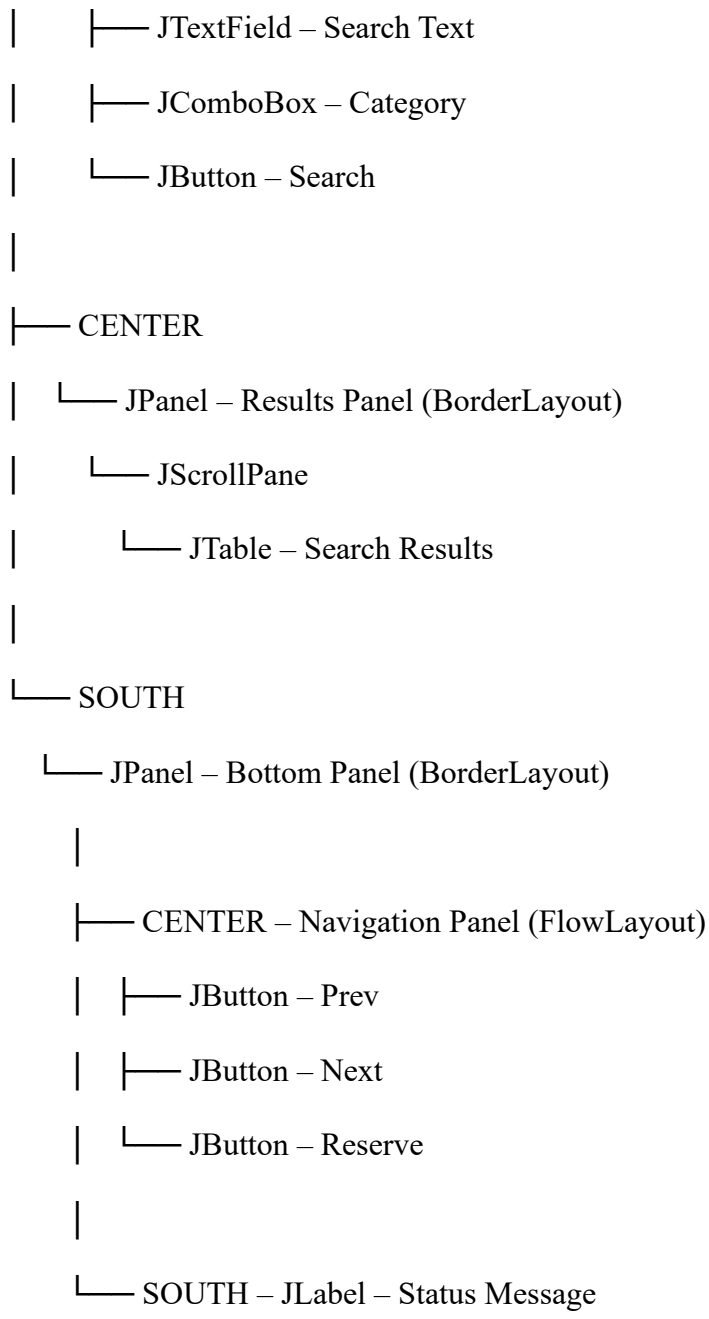
Nested Panel Structure

JFrame – Library Search

├── NORTH

│ ├── JLabel – Campus Library Search

│ └── JPanel – Search Panel (FlowLayout)



3.6 Task 3.3 – Implement the Screen

LibrarySearch.java

```
import java.awt.*;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;


public class LibrarySearch extends JFrame {
```

```
JTextField searchField = new JTextField(15);

JComboBox<String> category = new JComboBox<>(
    new String[]{"All", "Books", "Journals", "Authors"}
);

JButton searchButton = new JButton("Search");
JButton prevButton = new JButton("Prev");
JButton nextButton = new JButton("Next");
JButton reserveButton = new JButton("Reserve");

JLabel status = new JLabel("Ready");

JTable table;

public LibrarySearch() {

    setTitle("Campus Library Search");
    setLayout(new BorderLayout(5, 5));

    // Title panel
    JLabel title = new JLabel(
        "CAMPUS LIBRARY SEARCH",
        JLabel.CENTER
```

```
);

title.setFont(new Font("Arial", Font.BOLD, 20));

// Search panel
JPanel searchPanel = new JPanel(new FlowLayout());

searchPanel.add(new JLabel("Search"));
searchPanel.add(searchField);
searchPanel.add(category);
searchPanel.add(searchButton);

JPanel topPanel = new JPanel(new BorderLayout());
topPanel.add(title, BorderLayout.NORTH);
topPanel.add(searchPanel, BorderLayout.SOUTH);

add(topPanel, BorderLayout.NORTH);

// Results table
String[] columns = {
    "Book ID", "Book Name", "Author", "Category"
};

Object[][] data = {
    {"101", "Java Programming", "James", "Books"},
```

```
    {"102", "Computer Networks", "Andrew", "Books"},
    {"103", "Database Systems", "Ramakrishnan", "Books"}
};

table = new JTable(
    new DefaultTableModel(data, columns)
);

JScrollPane scrollPane = new JScrollPane(table);

JPanel resultsPanel = new JPanel(new BorderLayout());
resultsPanel.add(scrollPane, BorderLayout.CENTER);

add(resultsPanel, BorderLayout.CENTER);

// Bottom navigation
JPanel navigation = new JPanel(new FlowLayout());

navigation.add(prevButton);
navigation.add(nextButton);
navigation.add(reserveButton);

JPanel bottomPanel = new JPanel(new BorderLayout());

bottomPanel.add(navigation, BorderLayout.CENTER);
```

```
bottomPanel.add(status, BorderLayout.SOUTH);

add(bottomPanel, BorderLayout.SOUTH);

// Search action
searchButton.addActionListener(e -> {
    String value = searchField.getText();

    if (value.trim().isEmpty()) {
        status.setText("Enter a search term");
    } else {
        status.setText("Searching for: " + value);
    }
});

prevButton.addActionListener(e ->
    status.setText("Previous page selected")
);

nextButton.addActionListener(e ->
    status.setText("Next page selected")
);

reserveButton.addActionListener(e ->
    status.setText("Reserve button selected")
```

```
        );

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(700, 450);
        setLocationRelativeTo(null);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() ->
            new LibrarySearch().setVisible(true)
        );
    }
}
```

3.7 Task 3.4 – Test the Resizing Behaviour

The application must be tested at three different window sizes.

Test	Window Size	Expected Behaviour	Actual Result
TC01	500 × 350	Results area should be visible with limited space.	Record after execution
TC02	800 × 500	Results area should expand.	Record after execution
TC03	1100 × 700	Results area should occupy the larger centre region.	Record after execution

Screenshot Evidence

Attach three screenshots:

1. Small window showing the search row, results area, and bottom controls.

2. Medium window showing the expanded results area.
3. Large window showing the results area occupying the extra available space.

Observation: BorderLayout allows the centre region to grow when additional space is available. FlowLayout keeps the search and navigation components arranged in rows.

3.8 Alternative Concept – GridBagLayout and Absolute Positioning

GridBagLayout

GridBagLayout is a flexible layout manager that arranges components in a grid. Components can occupy different numbers of rows and columns and can expand according to constraints.

Absolute Positioning

Absolute positioning uses:

setLayout(null);

Components are positioned manually using:

component.setBounds(x, y, width, height);

Comparison

Feature	**Nested BorderLayout and FlowLayout**	**GridBagLayout**	**Absolute Positioning**
Component arrangement	Region-based and row-based	Flexible grid	Manually specified
Resizing	Good for centre-region expansion	Good with correct constraints	Poor without manual resizing logic
Code complexity	Low	Medium to high	Initially simple, but difficult to maintain

Layout control	Moderate	High	Complete manual control
Responsiveness	Good	Good when correctly configured	Usually poor

Complexity Analysis

Let n be the number of components.

- BorderLayout and FlowLayout perform layout calculations based on the components in their containers. Their layout work is generally proportional to the number of components being arranged.
- GridBagLayout also examines component constraints and calculates positions. Its layout work depends on the number of components and the complexity of the constraints.
- Absolute positioning avoids layout-manager calculations, but manual positioning and resizing logic must be written separately.

For space complexity, the main factor is the number of GUI components and containers. Nested panels introduce additional container objects, while a flatter design may use fewer containers. Actual memory usage also depends on the component types.

3.9 Conclusion

The campus library search screen uses BorderLayout and FlowLayout to achieve a flexible design. The centre results area expands when the window is resized, while the search and navigation rows remain organized. Nested containers make the interface easier to maintain compared with absolute positioning.

TASK 4 – EVENT-HANDLING FORENSICS

Bloom's Level: K4 – Analyse

7 Marks

A drawing pad is supposed to draw a dot wherever the user clicks and show the click count in a label. Users report that nothing is drawn and the count stays at 0.

```
public class DrawPad extends Frame {
  int count = 0; Label status = new Label("Clicks: 0");
  public DrawPad() { add(status,
  BorderLayout.SOUTH); addMouseListener(new
  MouseAdapter() { public void
  mouseclicked(MouseEvent e) { count++;
        Graphics g = getGraphics(); g.fillOval(e.getX(), e.getY(), 6, 6);
      }
    });
    setSize(400, 300); setVisible(true); }
}
```

4.1 Introduction

Event handling allows a Java application to respond to user interactions such as mouse clicks, keyboard input, and window actions.

A drawing pad application uses mouse events to draw dots on the window. The application must count the clicks and preserve the dots even after the window is minimized and restored.

4.2 Problem Statement

A drawing pad is expected to draw a dot wherever the user clicks and display the total click count in a label. However, the given program does not respond to mouse clicks correctly.

Even after correcting the mouse event method, the dots may disappear when the window is minimized and restored because the application does not preserve the drawing data.

The objective is to identify the errors and repair the program using a stored list of points and the paint() method.

4.3 Objectives

1. To identify mouse event-handling errors.
2. To understand the use of MouseAdapter.
3. To store mouse-click positions.
4. To redraw points during repainting.
5. To understand the purpose of the @Override annotation.

4.4 Task 4.1 – Locate the Defect

The given program contains two important defects.

Defect 1: Incorrect method name

The given code uses:

public void mouseclicked(MouseEvent e)

The correct method name is:

public void mouseClicked(MouseEvent e)

Java method names are case-sensitive. Because the method name is incorrect, the actual mouse-click event is not handled.

Defect 2: Label is not updated

The program increases the click count, but it does not update the label text. Therefore, the label continues to display:

Clicks: 0

The label must be updated using:

status.setText("Clicks: " + count);

4.5 Task 4.2 – Construct a Failing Interaction

The following interaction demonstrates the drawing problem:

1. Open the drawing pad.
2. Click several positions inside the window.
3. Observe the dots displayed on the screen.
4. Minimize the window.
5. Restore the window.
6. Observe that the dots may disappear.

Reason

The original program uses:

Graphics g = getGraphics();

g.fillOval(e.getX(), e.getY(), 6, 6);

This draws directly on the current screen. The drawing is not stored in the application.

When the window is minimized, restored, or repainted, Java redraws the window by calling its painting mechanism. Since the program has not stored the previous click positions, it cannot redraw the old dots.

4.6 Task 4.3 – Repair the Program

The correct solution stores every mouse-click position in an ArrayList of Point objects.

The paint() method then redraws all the stored points whenever the window needs repainting.

Complete Code – DrawPad.java

```
import java.awt.BorderLayout;
import java.awt.Frame;
import java.awt.Graphics;
import java.awt.Label;
import java.awt.Point;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.util.ArrayList;
import java.util.List;

public class DrawPad extends Frame {

    private final List<Point> points = new ArrayList<>();

    private final Label status = new Label("Clicks: 0");

    public DrawPad() {

        super("Draw Pad");
```

```
add(status, BorderLayout.SOUTH);

addMouseListener(new MouseAdapter() {

    // TODO 1 completed
    @Override
    public void mouseClicked(MouseEvent e) {

        points.add(e.getPoint());

        status.setText(
            "Clicks: " + points.size()
        );

        repaint();
    }
});

addWindowListener(new WindowAdapter() {
```

```
        @Override
        public void windowClosing(WindowEvent e) {
            dispose();
        }
    });

    setSize(400, 300);
}

@Override
public void paint(Graphics g) {

    // TODO 2 completed
    for (Point p : points) {

        g.fillOval(
            p.x - 3,
            p.y - 3,
            6,
            6
        );
```

```
        }
    }

    public static void main(String[] args) {
        new DrawPad().setVisible(true);
    }
}
```

Explanation of TODO 1

1. The mouse click is detected using mouseClicked().
2. The click position is stored using points.add(e.getPoint()).
3. The label is updated using the number of stored points.
4. repaint() requests the window to be redrawn.

Explanation of TODO 2

The paint() method loops through all stored points. For every point, it draws a circle with a width and height of six pixels.

The position is adjusted using p.x - 3 and p.y - 3 so that the circle is centred around the click location.

4.7 Task 4.4 – Prevent Recurrence Using @Override

The @Override annotation informs the compiler that a method is intended to override a method from a superclass or implement a method from an interface.

Correct:

```
@Override
```

```
public void mouseClicked(MouseEvent e) {
    // Event handling
}
```

Incorrect:

```
@Override
public void mouseclicked(MouseEvent e) {
    // Wrong spelling
}
```

If the method is misspelled, the compiler reports an error because it does not override the expected method.

Without @Override, the incorrect method may compile as an unrelated method, leaving the actual mouse event unhandled.

4.8 Required Screenshot Evidence

The assignment requires screenshots before and after minimizing the window.

Screenshot	**What It Must Demonstrate**
Screenshot 1	Drawing pad opened with no clicks
Screenshot 2	Multiple clicks and updated click count
Screenshot 3	Window minimized and restored, with dots still visible
Screenshot 4	Compiler error after removing @Override and misspelling mouseClicked

4.9 Alternative Concept – BufferedImage

Instead of storing each point in an ArrayList and redrawing all points, an off-screen BufferedImage can be used.

The application draws each new dot onto the image. The paint() method copies the image onto the screen.

Example Concept

```
BufferedImage image =
   new BufferedImage(
      400, 300, BufferedImage.TYPE_INT_ARGB
   );
```

A Graphics object can be obtained from the image:

```
Graphics g = image.getGraphics();
g.fillOval(x - 3, y - 3, 6, 6);
g.dispose();
```

During painting:

```
g.drawImage(image, 0, 0, this);
```

Comparison

Feature	**ArrayList of Points**	**BufferedImage**
Stored information	Coordinates of clicks	Pixel data of the drawing
Space	O(n) points	O(width × height) pixels

Drawing a new dot	Add one point	Draw onto image
Repaint operation	Redraw all n points	Copy the image
Suitable for	Simple dot drawing	Larger drawings and image-based drawing

Let n represent the number of clicks.

- ArrayList approach: repainting requires O(n) drawing operations.
- BufferedImage approach: repainting copies the stored image, which processes the image's pixel area.

4.10 Conclusion

The drawing pad program fails because the mouse-click method is incorrectly named and the label is not updated. Drawing directly using getGraphics() does not preserve the drawing during repainting. Storing points in an ArrayList and drawing them inside paint() provides a reliable solution. The @Override annotation helps identify method-name errors during compilation.

TASK 5 – SAME SCREEN, DIFFERENT EVENT-HANDLING STYLES

Bloom's Level: K5 – Evaluate

7 Marks

A simple currency-converter window has an amount field, a From/To currency selection, a Convert button and a result label.

5.1 Introduction

Java provides different ways to handle events in GUI applications. An event listener can be implemented by the main frame, a named inner class, an anonymous inner class, or a lambda expression.

A currency converter is used to demonstrate these four event-handling styles. The application accepts an amount in Indian Rupees (INR), allows the user to select USD or EUR, and displays the converted value.

5.2 Problem Statement

A currency converter window must contain an amount field, currency selection, Convert button, and result label.

The application must be implemented using four event-handling approaches:

1. Frame implementing ActionListener.
2. Named inner class.
3. Anonymous inner class.
4. Lambda expression.

The application must also be modified to support the Enter key and display the result in a dialog box.

5.3 Objectives

1. To understand different event-handling styles.
2. To implement ActionListener in a JFrame.
3. To use named and anonymous inner classes.
4. To implement event handling using lambda expressions.
5. To compare the readability and maintainability of the four approaches.

5.4 Task 5.1 – Four Implementations

(a) Frame Implements ActionListener

In this approach, the main frame implements the ActionListener interface. The actionPerformed() method handles the button click.

```
import java.awt.FlowLayout;

import java.awt.event.*;

import javax.swing.*;
```

```
public class ConverterA extends JFrame
        implements ActionListener {

    JTextField amount = new JTextField(10);

    JComboBox<String> target =
        new JComboBox<>(new String[]{"USD", "EUR"});

    JButton button = new JButton("Convert");

    JLabel result = new JLabel(" ");

    public ConverterA() {

        setTitle("Currency Converter");
        setLayout(new FlowLayout());

        add(new JLabel("INR"));
        add(amount);
        add(target);
        add(button);
        add(result);

        button.addActionListener(this);
```

```
    addWindowListener(new WindowAdapter() {
        @Override
        public void windowClosing(WindowEvent e) {
            dispose();
        }
    });

    setSize(350, 180);
    setDefaultCloseOperation(EXIT_ON_CLOSE);
}

@Override
public void actionPerformed(ActionEvent e) {

    try {
        double value =
            Double.parseDouble(amount.getText());

        String currency =
            (String) target.getSelectedItem();

        double rate =
            currency.equals("USD") ? 0.012 : 0.011;
```

```
            double converted = value * rate;


            result.setText(
                String.format("%.2f %s", converted, currency)
            );


        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
        }
    }


    public static void main(String[] args) {
        SwingUtilities.invokeLater(() ->
            new ConverterA().setVisible(true)
        );
    }
}
```

Explanation: The JFrame itself receives the button event because it implements ActionListener. This is simple for small programs but can make large classes difficult to maintain when many events are handled in the same method.

(b) Named Inner Class

A named inner class is defined inside another class and implements the required listener interface.

```
import java.awt.FlowLayout;

import java.awt.event.*;
```

```
import javax.swing.*;

public class ConverterB extends JFrame {

    JTextField amount = new JTextField(10);

    JComboBox<String> target =
        new JComboBox<>(new String[]{"USD", "EUR"});

    JButton button = new JButton("Convert");

    JLabel result = new JLabel(" ");

    public ConverterB() {

        setTitle("Currency Converter");
        setLayout(new FlowLayout());

        add(new JLabel("INR"));
        add(amount);
        add(target);
        add(button);
        add(result);

        button.addActionListener(new ConvertHandler());
```

```
    addWindowListener(new WindowAdapter() {
        @Override
        public void windowClosing(WindowEvent e) {
            dispose();
        }
    });

    setSize(350, 180);
    setDefaultCloseOperation(EXIT_ON_CLOSE);
}

class ConvertHandler implements ActionListener {

    @Override
    public void actionPerformed(ActionEvent e) {

        try {
            double value =
                Double.parseDouble(amount.getText());

            String currency =
                (String) target.getSelectedItem();

            double rate =
```

```
                currency.equals("USD") ? 0.012 : 0.011;

            result.setText(
                String.format(
                    "%.2f %s", value * rate, currency
                )
            );

        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
        }
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() ->
        new ConverterB().setVisible(true)
    );
}
}
```

Explanation: The event-handling logic is separated into a named inner class. The inner class can access the fields of the outer JFrame directly. It is useful when the handler contains more code or must be reused.

(c) Anonymous Inner Class

An anonymous inner class is created without defining a separate named class.

```
import java.awt.FlowLayout;
import java.awt.event.*;
import javax.swing.*;

public class ConverterC extends JFrame {

    JTextField amount = new JTextField(10);

    JComboBox<String> target =
        new JComboBox<>(new String[]{"USD", "EUR"});

    JButton button = new JButton("Convert");

    JLabel result = new JLabel(" ");

    public ConverterC() {

        setTitle("Currency Converter");
        setLayout(new FlowLayout());

        add(new JLabel("INR"));
        add(amount);
        add(target);
        add(button);
        add(result);
```

```
button.addActionListener(new ActionListener() {

    @Override
    public void actionPerformed(ActionEvent e) {

        try {
            double value =
                Double.parseDouble(amount.getText());

            String currency =
                (String) target.getSelectedItem();

            double rate =
                currency.equals("USD") ? 0.012 : 0.011;

            result.setText(
                String.format(
                    "%.2f %s", value * rate, currency
                )
            );

        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
        }
```

```
        }
    });

    addWindowListener(new WindowAdapter() {
        @Override
        public void windowClosing(WindowEvent e) {
            dispose();
        }
    });

    setSize(350, 180);
    setDefaultCloseOperation(EXIT_ON_CLOSE);
  }

  public static void main(String[] args) {
    SwingUtilities.invokeLater(() ->
        new ConverterC().setVisible(true)
    );
  }
}
```

Explanation: The listener is created directly inside addActionListener(). It avoids creating a separate named handler class. It is useful for a small event-handling operation that is needed only once.

(d) Lambda Expression

A lambda expression provides a shorter way to implement a functional interface such as ActionListener.

The assignment provides a partially completed lambda version. The completed version is given below.

Complete Code – Converter.java

```
import java.awt.FlowLayout;

import java.awt.event.ActionListener;

import javax.swing.JButton;

import javax.swing.JComboBox;

import javax.swing.JFrame;

import javax.swing.JLabel;

import javax.swing.JTextField;

import javax.swing.SwingUtilities;

import javax.swing.JOptionPane;

import java.awt.event.WindowAdapter;

import java.awt.event.WindowEvent;


public class Converter extends JFrame {


   private final JTextField amount = new JTextField(10);


   private final JComboBox<String> target =

      new JComboBox<>(new String[]{"USD", "EUR"});


   private final JLabel result = new JLabel(" ");
```

```
private static double rate(String currency) {
    return currency.equals("USD") ? 0.012 : 0.011;
}

public Converter() {

    super("Currency Converter");

    setLayout(new FlowLayout());

    JButton convertButton = new JButton("Convert");

    add(new JLabel("INR"));
    add(amount);
    add(target);
    add(convertButton);
    add(result);

    // TODO 1 completed
    ActionListener convertAction = e -> convert();

    convertButton.addActionListener(convertAction);
    amount.addActionListener(convertAction);
```

```
    addWindowListener(new WindowAdapter() {
        @Override
        public void windowClosing(WindowEvent e) {
            dispose();
        }
    });

    setDefaultCloseOperation(EXIT_ON_CLOSE);

    pack();
    setLocationRelativeTo(null);
}

private void convert() {

    // TODO 2 completed
    try {
        double value =
            Double.parseDouble(amount.getText());

        String currency =
            (String) target.getSelectedItem();

        double converted =
            value * rate(currency);
```

```
            String message =
                String.format("%.2f %s", converted, currency);

            result.setText(message);

            JOptionPane.showMessageDialog(this, message);

        } catch (NumberFormatException e) {

            result.setText("Invalid amount");

            JOptionPane.showMessageDialog(
                this, "Invalid amount"
            );
        }
    }

    public static void main(String[] args) {

        SwingUtilities.invokeLater(() ->
            new Converter().setVisible(true)
        );
    }
}
```

Explanation of TODO 1

One lambda listener is created:

ActionListener convertAction = e -> convert();

The same listener is registered with both the Convert button and the amount text field. Pressing the button or pressing Enter in the text field invokes the same conversion method.

Explanation of TODO 2

1. The amount is read from the text field.
2. Double.parseDouble() converts the input into a number.
3. The selected currency is retrieved.
4. The appropriate conversion rate is selected.
5. The converted amount is calculated.
6. The result is displayed with two decimal places.
7. If the input is not numeric, an Invalid amount message is displayed.

Note: The conversion rates are the fixed sample values supplied in the assignment. They are not live exchange rates.

5.5 Task 5.2 – Change the Requirement

The modified application must:

- Convert when the user clicks the Convert button.
- Convert when the user presses Enter in the amount field.
- Display the converted result in a dialog box.
- Display an error dialog for an invalid amount.

Changes Required in Each Implementation

Implementation	Changes Required

Frame implements ActionListener	Register the frame as an ActionListener for the text field also. Handle the event in the existing actionPerformed() method.
Named inner class	Register the same named handler object with both components.
Anonymous inner class	Reuse a single listener object, or add a second anonymous listener with the same conversion logic.
Lambda expression	Store the lambda in one ActionListener variable and register it with both components.

Example of Shared Listener

ActionListener action = e -> convert();

button.addActionListener(action);

amount.addActionListener(action);

This avoids repeating the same event-handling logic.

5.6 Task 5.3 – Compare the Four Styles

Criteria	**Frame Listener**	**Named Inner Class**	**Anonymous Inner Class**	**Lambda**
Readability	Simple for small programs	Good for larger handlers	Good for short handlers	Very concise
Access to frame fields	Direct	Direct	Direct	Can access enclosing fields
Handler reuse	Limited unless structured separately	Good	Requires storing the object for reuse	Good when stored in a variable

Code length	Moderate	More code	Moderate	Short
Maintenance	Can become difficult in large frames	Good separation	Suitable for small handlers	Good for simple actions

Which Style Is Suitable for a Large Application?

For a large application, a named inner class or a separate reusable Action class is suitable because it separates the event-handling logic from the GUI construction.

A lambda expression is convenient for small operations. However, when the application has complex validation, multiple actions, or reusable business logic, separating the handlers into named classes improves organization.

5.7 Alternative Concept – Method Reference and Shared Action

A method reference can be used instead of a lambda when the event handler simply calls an existing method.

Example:

```
button.addActionListener(e -> convert());
```

Can be written as:

```
button.addActionListener(e -> convert());
```

For a method that accepts no event argument, a small adapter lambda is needed. A direct this::convert reference is only compatible when the method signature matches the listener's abstract method.

A shared Swing Action can be used by a button, menu item, and keyboard shortcut.

Example:

```
Action convertAction = new AbstractAction("Convert") {
   @Override
   public void actionPerformed(ActionEvent e) {
      convert();
   }
```

};

button.setAction(convertAction);

The same Action object can be connected to multiple GUI triggers.

Comparison

Approach	Listener/Action Objects	Adding a New Trigger
Separate listeners	May create multiple listener objects	Register another listener
Shared lambda	One shared listener object	Register the same listener
Method reference	Method-reference listener	Register the reference
Shared Swing Action	One shared Action object	Attach the Action to another component

5.8 Conclusion

Java provides four useful approaches to event handling. Implementing ActionListener in the frame is suitable for simple applications, while named inner classes provide better separation. Anonymous inner classes are useful for one-time handlers, and lambda expressions reduce code length. Shared listeners and Swing Actions make it easier to support multiple triggers without duplicating logic.

TASK 6 – DESIGN CHALLENGE

Bloom's Level: K5 – Evaluate

Campus Event Registration Desktop Application

13 Marks

The cultural committee needs a desktop application to manage event registrations. Students register for events through a form; the committee views all registrations in a table, filters them by event and exports the list. The data is stored in a MySQL database.

The application must remain responsive while data is loading, must reject invalid input clearly, and must not allow a student to register twice for the same event.

You are not given the design.

6.1 Introduction

A Campus Event Registration Desktop Application is used to manage student registrations for college events. Students can register for cultural, technical, sports, and other campus activities.

The application provides a graphical interface where students enter their details and select an event. The committee can view registrations, filter them by event, and export registration information.

Java Swing is used to develop the graphical interface, while JDBC is used to connect the application to a MySQL database.

The application must validate user input, prevent duplicate registrations, and remain responsive while database operations are performed.

6.2 Problem Statement

The cultural committee requires a desktop application to manage student registrations for campus events.

The application must allow students to register for events and allow committee members to view and filter registrations. Registration information must be stored in a MySQL database.

The application must satisfy the following requirements:

1. Students must be able to register for campus events.
2. The committee must be able to view registrations in a table.
3. The committee must be able to filter registrations by event.
4. The application must support exporting registration details.
5. Invalid inputs must be rejected with meaningful messages.
6. A student must not be allowed to register twice for the same event.

7. Database operations must not freeze the GUI.

6.3 Objectives

1. To design a desktop application using Java Swing.
2. To integrate a Java application with MySQL using JDBC.
3. To implement input validation.
4. To prevent duplicate event registrations.
5. To display registration details using JTable.
6. To implement event filtering and data export.
7. To use SwingWorker for background database operations.

6.4 Stage 1 – Interpret the Problem

A. Users

The application has two main types of users.

1. Students

- Enter their student ID.
- Enter their name.
- Select an event.
- Submit the registration.
- Receive a confirmation or error message.

2. Event Committee Members

- View all registrations.
- Filter registrations by event.
- Check student registration details.
- Export registration data.
- Monitor the total registrations.

B. Screens

The application contains the following screens:

Screen	Purpose
Registration Screen	Allows students to register for an event.
Registration List Screen	Displays all student registrations.
Filter Screen	Filters registrations by event.
Export Option	Exports registration details to a file.
Confirmation Dialog	Displays successful registration messages.
Error Dialog	Displays validation and database errors.

C. Data Required

The system stores the following information:

Field	Data Type	Description
Registration ID	INT	Unique registration number
Student ID	VARCHAR	Student's register number
Student Name	VARCHAR	Student's name
Event Name	VARCHAR	Selected event
Department	VARCHAR	Student's department
Registration Date	TIMESTAMP	Date and time of registration

D. Business Rules

1. Student ID cannot be empty.
2. Student name cannot be empty.
3. An event must be selected.
4. A student can register only once for the same event.
5. Different students can register for the same event.
6. A student can register for different events.

7. Registration information must be saved in the database.
8. The committee must be able to filter registrations.
9. Database errors must be displayed clearly.
10. The GUI must remain responsive during database operations.

E. Testable Requirements

Requirement ID	Requirement	Expected Result
R01	Enter valid student details	Registration accepted
R02	Leave student ID empty	Validation error displayed
R03	Leave student name empty	Validation error displayed
R04	Do not select an event	Validation error displayed
R05	Register the same student for the same event twice	Second registration rejected
R06	Register a student for another event	Registration accepted if valid
R07	Filter by event	Only matching registrations displayed
R08	Export registration details	File created with registration data
R09	Database connection fails	Error displayed without freezing the GUI

6.5 Stage 2 – Propose the Screen and Event Flow

A. Registration Screen

The registration screen contains:

- Student ID text field.
- Student name text field.

- Department selection.
- Event selection.
- Register button.
- Clear button.
- Status label.

B. Registration List Screen

No.	Event Name	Event Location	Event Category	Event Date	Duration	Participants / Capacity	Registration Period	Application (Details)
1	Education Seminar	Education Seminar	Education	2010-11-01 to 2010-11-03	3 days	0/20	2010-10-04 to 2010-10-29	Apply
2	Open Community	Joseon Hotel	Seminar	2010-10-04 to 2010-10-06	3 days	1/100	2010-09-27 to 2010-09-30	Approved

3	**Test Approval**	**City Hall**	**Education**	**2010-09-22 to 2010-09-30**	**9 days**	**1/12**	**2010-09-07 to 2010-09-17**	**Approved**
4	**Unclear text**	**Unclear text**	**Education**	**2010-09-21 to 2010-09-30**	**10 days**	**1/1**	**2010-09-01 to 2010-09-10**	**Approved**
5	**Unclear text**	**Unclear text**	**Education**	**2010-09-17 to 2010-09-17**	**1 day**	**1/0**	**2010-09-10 to 2010-09-10**	**Approved**
6	**12r**	**12r**	**Education**	**2010-09-30**	**1 day**	**1/0**	**2010-09-09 to 2010-09-09**	**Approved**

				to 2010-09-30				
7	124	3q	Education	2010-09-28 to 2010-09-28	1 day	1/0	2010-09-07 to 2010-09-07	Approved
8	124	24124	Education	2010-09-09 to 2010-09-10	2 days	1/0	2010-09-08 to 2010-09-08	Approved
9	124	124	Education	2010-09-08 to 2010-	1 day	1/124	2010-09-08 to 2010-09-08	Returned

				09-08				
10	**12**	**124**	**Education**	**2010-09-23 to 2010-09-24**	**2 days**	**1/0**	**2010-09-10 to 2010-09-10**	**Returned**

The registration list screen contains:

- Registration table.
- Event filter.
- Filter button.
- Refresh button.
- Export button.
- Status label.

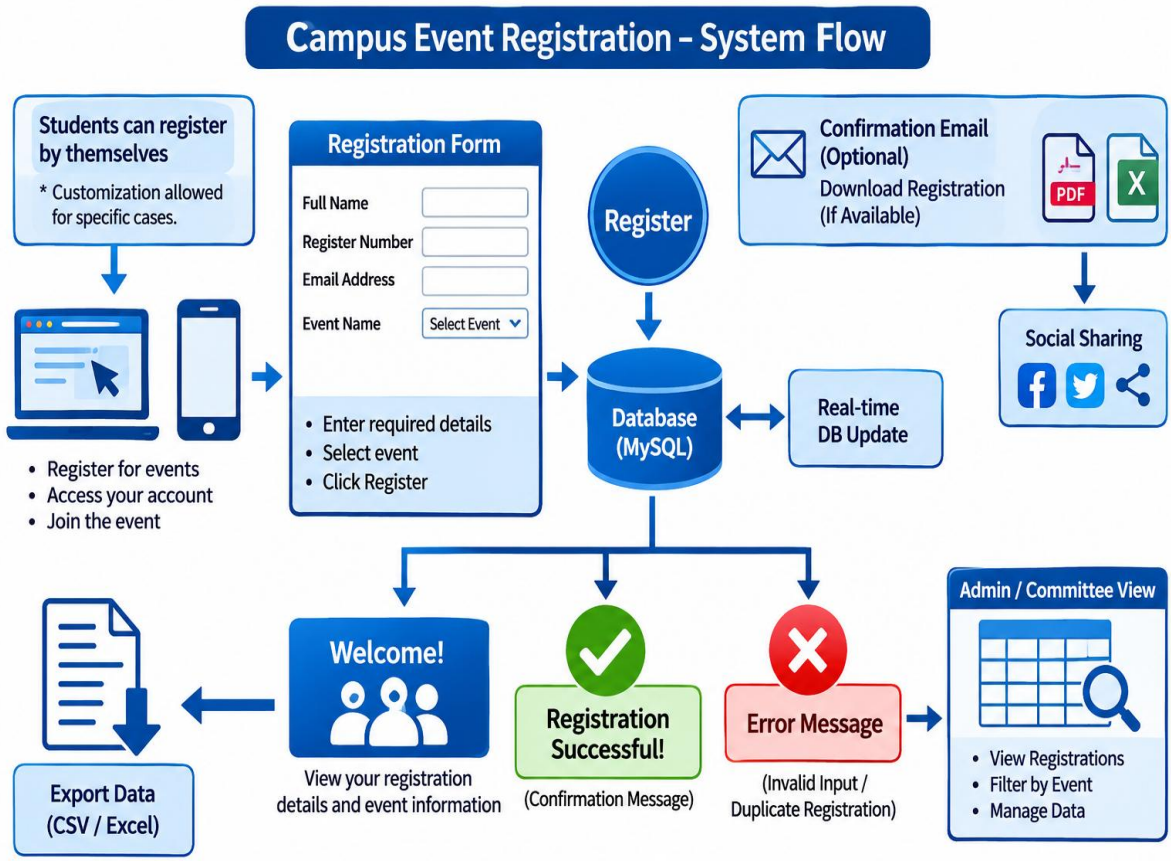
C. Menu Bar

The application must contain a menu bar.

Menu	**Menu Item**	**Action**
File	New Registration	Opens the registration form
File	Export	Exports registration details
File	Exit	Closes the application
View	Refresh	Reloads registration data

Help	About	Displays application information

D. Screen Flow Diagram



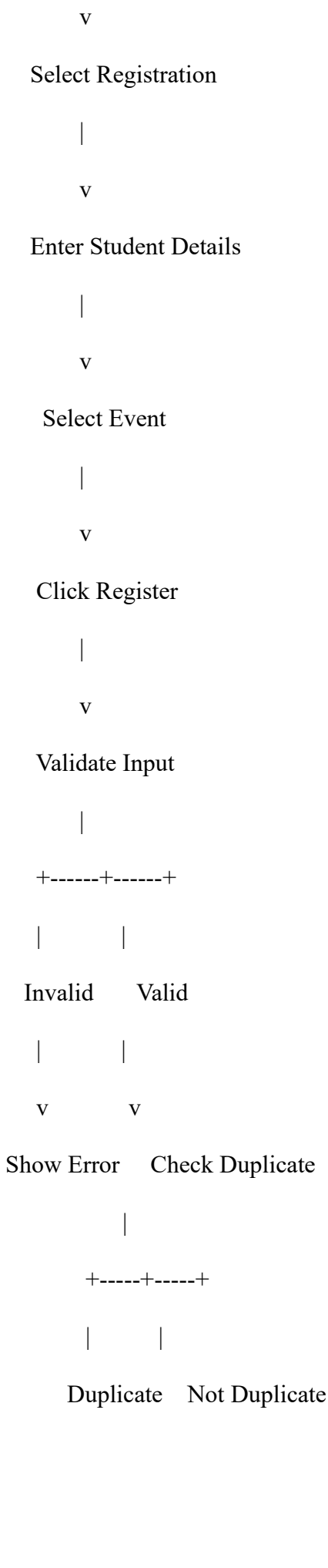
START

|

v

Open Main Window

|


v
Select Registration
|
v
Enter Student Details
|
v
Select Event
|
v
Click Register
|
v
Validate Input
|
+------+------+
| |
Invalid Valid
| |
v v
Show Error Check Duplicate
|
+-----+-----+
| |
Duplicate Not Duplicate

| |

v v

Show Error Save to MySQL

|

v

Show Confirmation

|

v

END / Continue

E. Event Flow

Event Source	Event Object	Listener	Action
Register button	ActionEvent	ActionListener	Validate and save registration
Clear button	ActionEvent	ActionListener	Clear all input fields
Event ComboBox	ActionEvent	ActionListener	Update selected event
Filter button	ActionEvent	ActionListener	Load registrations for selected event
Refresh button	ActionEvent	ActionListener	Reload registration table
Export button	ActionEvent	ActionListener	Export table data
Exit menu	ActionEvent	ActionListener	Close application
Window closing	WindowEvent	WindowAdapter	Confirm application closing

6.6 Stage 3 – Identify the Critical Design Decisions

A. Input Validation

Input validation must be performed before saving information to the database.

The following checks are required:

- Student ID must not be empty.
- Student name must not be empty.
- Department must be selected.
- Event must be selected.
- Invalid data must be rejected.
- Error messages must be understandable.

Example:

```
if (studentId.trim().isEmpty()) {
   JOptionPane.showMessageDialog(
      this, "Enter Student ID"
   );
   return;
}
```

B. Duplicate Checking

A student must not register twice for the same event.

The application can check whether a record already exists using:

```
SELECT COUNT(*)
FROM registrations
WHERE student_id = ?
AND event_name = ?;
```

However, checking before insertion alone is not sufficient when multiple requests occur simultaneously. The database must also enforce a unique constraint on the student ID and event combination.

Example:

UNIQUE (student_id, event_name)

This ensures that duplicate registrations are prevented even if two requests attempt to insert the same registration at nearly the same time.

C. Database Access

JDBC is used to communicate with MySQL.

The application uses:

- Connection to connect to the database.
- PreparedStatement to execute parameterized SQL.
- ResultSet to retrieve records.
- SQLException to handle database errors.

D. Keeping the GUI Responsive

Database operations may take time. If they are performed directly inside an event listener, the GUI may freeze.

SwingWorker is used to perform long-running operations in a background thread.

Example:

```
new SwingWorker<Void, Void>() {

    @Override
    protected Void doInBackground() throws Exception {

        // Perform database operation here
        return null;
    }

    @Override
```

```
    protected void done() {


        // Update GUI after background work
    }


}.execute();
```

The doInBackground() method performs the database work. The done() method updates the GUI after the operation finishes.

6.7 Stage 4 – Design the Classes and Handlers

A. Proposed Class Structure



The application is divided into three main layers.

1. User Interface Layer

Class: EventRegistrationGUI

Responsibilities:

- Displays the registration form.
- Accepts student details.
- Displays the JTable.
- Provides filtering and export options.
- Displays success and error dialogs.

- Handles user actions.

2. Service Layer

Class: RegistrationService

Responsibilities:

- Validates student information.
- Checks registration rules.
- Processes registration requests.
- Prevents duplicate registrations.
- Communicates with the DAO layer.

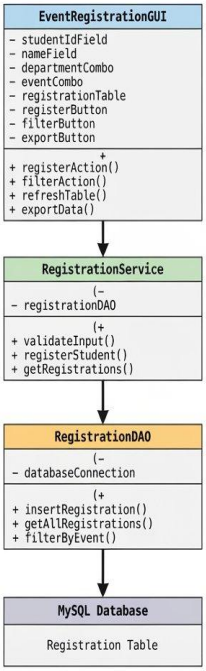
3. Data Access Layer

Class: RegistrationDAO

Responsibilities:

- Establishes the MySQL connection.
- Inserts registration records.
- Retrieves registration records.
- Filters records by event.
- Handles SQL exceptions.

B. Class Diagram



C. Pseudocode for Register Action

ALGORITHM: REGISTER_STUDENT

BEGIN

READ studentID

READ studentName

READ department

READ eventName

IF studentID is empty THEN

DISPLAY "Enter Student ID"

```
    STOP

END IF

IF studentName is empty THEN

    DISPLAY "Enter Student Name"
    STOP

END IF

IF department is not selected THEN

    DISPLAY "Select Department"
    STOP

END IF

IF eventName is not selected THEN

    DISPLAY "Select Event"
    STOP

END IF
```

```
    VALIDATE student details

    CHECK duplicate registration

    IF duplicate exists THEN

        DISPLAY "Already Registered"

    ELSE

        SAVE registration in database

        DISPLAY "Registration Successful"

    END IF

END
```

D. Pseudocode for Filter Action

```
ALGORITHM: FILTER_REGISTRATIONS

BEGIN

    READ selectedEvent

    IF selectedEvent is empty THEN
```

```
    DISPLAY "Select an Event"
    STOP

  END IF

  FETCH registrations
    WHERE eventName = selectedEvent

  CLEAR existing table rows

  FOR EACH registration IN result

    ADD registration to JTable

  END FOR

  DISPLAY "Filter completed"

END
```

6.8 Stage 5 – Implement and Demonstrate

The following implementation demonstrates the required Swing components, JDBC connection, input validation, duplicate prevention, table display, filtering, exporting, and background loading.

A. MySQL Database Creation

Create a database using MySQL.

SQL Script – database.sql

```
CREATE DATABASE campus_events;


USE campus_events;


CREATE TABLE registrations (


   registration_id INT AUTO_INCREMENT PRIMARY KEY,


   student_id VARCHAR(30) NOT NULL,


   student_name VARCHAR(100) NOT NULL,


   department VARCHAR(100) NOT NULL,


   event_name VARCHAR(100) NOT NULL,


   registration_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,


   UNIQUE (student_id, event_name)
);
```

Explanation:

- registration_id uniquely identifies each registration.
- student_id stores the student's register number.

- student_name stores the student's name.
- department stores the department name.
- event_name stores the selected event.
- registration_date records the registration time.
- The UNIQUE constraint prevents a student from registering for the same event twice.

B. JDBC Connection – DBConnection.java

```
import java.sql.Connection;

import java.sql.DriverManager;

import java.sql.SQLException;


public class DBConnection {


   private static final String URL =

      "jdbc:mysql://localhost:3306/campus_events";


   private static final String USER = "root";


   private static final String PASSWORD = "password";


   public static Connection getConnection()

         throws SQLException {


      return DriverManager.getConnection(

         URL, USER, PASSWORD
```

```
        );
    }
}
```

Replace the username and password with the credentials configured in your MySQL installation. Add the MySQL Connector/J driver to the Java project's classpath.

C. Registration Model – Registration.java

```
public class Registration {

    private String studentId;
    private String studentName;
    private String department;
    private String eventName;

    public Registration(
            String studentId,
            String studentName,
            String department,
            String eventName) {

        this.studentId = studentId;
        this.studentName = studentName;
        this.department = department;
        this.eventName = eventName;
    }
```

```
    public String getStudentId() {
        return studentId;
    }

    public String getStudentName() {
        return studentName;
    }

    public String getDepartment() {
        return department;
    }

    public String getEventName() {
        return eventName;
    }
}
```

D. Data Access Class – RegistrationDAO.java

```
import java.sql.*;
import java.util.*;

public class RegistrationDAO {

    public void insertRegistration(Registration r)
            throws SQLException {
```

```
    String sql =
        "INSERT INTO registrations " +
        "(student_id, student_name, department, event_name) " +
        "VALUES (?, ?, ?, ?)";

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)
    ) {

        ps.setString(1, r.getStudentId());
        ps.setString(2, r.getStudentName());
        ps.setString(3, r.getDepartment());
        ps.setString(4, r.getEventName());

        ps.executeUpdate();
    }
}

public List<Registration> getRegistrations(String event)
        throws SQLException {

    List<Registration> list = new ArrayList<>();

    String sql =
```

```
    "SELECT student_id, student_name, department, event_name " +
    "FROM registrations " +
    "WHERE (? = 'All' OR event_name = ?) " +
    "ORDER BY registration_id DESC";

try (
    Connection con = DBConnection.getConnection();
    PreparedStatement ps = con.prepareStatement(sql)
) {

    ps.setString(1, event);
    ps.setString(2, event);

    try (ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {

            Registration r = new Registration(
                rs.getString("student_id"),
                rs.getString("student_name"),
                rs.getString("department"),
                rs.getString("event_name")
            );

            list.add(r);
```

```
            }
        }
    }

    return list;
  }
}
```

E. Service Class – RegistrationService.java

```
import java.sql.SQLException;
import java.util.List;

public class RegistrationService {

  private RegistrationDAO dao =
    new RegistrationDAO();

  public void registerStudent(Registration r)
      throws Exception {

    if (r.getStudentId() == null ||
      r.getStudentId().trim().isEmpty()) {

      throw new IllegalArgumentException(
        "Student ID is required"
      );
```

```
}

if (r.getStudentName() == null ||
    r.getStudentName().trim().isEmpty()) {

    throw new IllegalArgumentException(
        "Student name is required"
    );
}

if (r.getDepartment() == null ||
    r.getEventName() == null) {

    throw new IllegalArgumentException(
        "Select department and event"
    );
}

try {
    dao.insertRegistration(r);

} catch (SQLException e) {

    // MySQL duplicate-entry error code
    if (e.getErrorCode() == 1062) {
```

```
                throw new Exception(
                    "Student already registered for this event"
                );
            }

            throw e;
        }
    }

    public List<Registration> getRegistrations(String event)
            throws SQLException {

        return dao.getRegistrations(event);
    }
}
```

F. Main GUI – EventRegistrationGUI.java

```
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.util.List;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;

public class EventRegistrationGUI extends JFrame {
```

```
JTextField studentId = new JTextField(12);
JTextField studentName = new JTextField(12);

JComboBox<String> department =
    new JComboBox<>(new String[]{
        "CSE", "ECE", "EEE", "MECH", "CIVIL"
    });

JComboBox<String> event =
    new JComboBox<>(new String[]{
        "Cultural Fest",
        "Technical Fest",
        "Sports Meet"
    });

JComboBox<String> filter =
    new JComboBox<>(new String[]{
        "All",
        "Cultural Fest",
        "Technical Fest",
        "Sports Meet"
    });

JButton register = new JButton("Register");
JButton clear = new JButton("Clear");
```

```
JButton refresh = new JButton("Refresh");

JButton filterButton = new JButton("Filter");

JButton export = new JButton("Export");


JLabel status = new JLabel("Ready");


DefaultTableModel model =

    new DefaultTableModel(

        new String[]{

            "Student ID",

            "Student Name",

            "Department",

            "Event"

        }, 0

    );


JTable table = new JTable(model);


RegistrationService service =

    new RegistrationService();


public EventRegistrationGUI() {


    setTitle("Campus Event Registration");
```

```
setLayout(new BorderLayout(5, 5));

// Registration form
JPanel form = new JPanel(new GridLayout(5, 2, 5, 5));

form.add(new JLabel("Student ID"));
form.add(studentId);

form.add(new JLabel("Student Name"));
form.add(studentName);

form.add(new JLabel("Department"));
form.add(department);

form.add(new JLabel("Event"));
form.add(event);

form.add(register);
form.add(clear);

// Table area
JScrollPane scroll =
    new JScrollPane(table);

// Filter panel
```

```
JPanel filterPanel = new JPanel(new FlowLayout());

filterPanel.add(new JLabel("Filter Event"));
filterPanel.add(filter);
filterPanel.add(filterButton);
filterPanel.add(refresh);
filterPanel.add(export);

// Main content
JPanel topPanel = new JPanel(new BorderLayout());

topPanel.add(form, BorderLayout.NORTH);
topPanel.add(filterPanel, BorderLayout.SOUTH);

add(topPanel, BorderLayout.NORTH);
add(scroll, BorderLayout.CENTER);
add(status, BorderLayout.SOUTH);

// Menu bar
JMenuBar menuBar = new JMenuBar();

JMenu fileMenu = new JMenu("File");

JMenuItem exportItem = new JMenuItem("Export");
JMenuItem exitItem = new JMenuItem("Exit");
```

```
fileMenu.add(exportItem);
fileMenu.add(exitItem);

menuBar.add(fileMenu);

setJMenuBar(menuBar);

// Register event
register.addActionListener(e -> registerStudent());

// Clear event
clear.addActionListener(e -> clearFields());

// Refresh event
refresh.addActionListener(e -> loadData("All"));

// Filter event
filterButton.addActionListener(e ->
    loadData((String) filter.getSelectedItem())
);

// Export event
export.addActionListener(e -> exportData());
```

```
exportItem.addActionListener(e -> exportData());

exitItem.addActionListener(e -> dispose());

// Window closing
addWindowListener(new WindowAdapter() {
    @Override
    public void windowClosing(WindowEvent e) {
        int choice = JOptionPane.showConfirmDialog(
            EventRegistrationGUI.this,
            "Do you want to exit?",
            "Confirm Exit",
            JOptionPane.YES_NO_OPTION
        );

        if (choice == JOptionPane.YES_OPTION) {
            dispose();
        }
    }
});

setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);

setSize(850, 550);
setLocationRelativeTo(null);
```

```
    loadData("All");
}

private void registerStudent() {

    String id = studentId.getText().trim();
    String name = studentName.getText().trim();

    String dept =
        (String) department.getSelectedItem();

    String selectedEvent =
        (String) event.getSelectedItem();

    Registration r =
        new Registration(id, name, dept, selectedEvent);

    register.setEnabled(false);

    new SwingWorker<Void, Void>() {

        @Override
        protected Void doInBackground() throws Exception {
```

```
        service.registerStudent(r);

        return null;
    }

    @Override
    protected void done() {

        register.setEnabled(true);

        try {
            get();

            JOptionPane.showMessageDialog(
                EventRegistrationGUI.this,
                "Registration Successful"
            );

            status.setText("Registration completed");

            clearFields();

            loadData("All");

        } catch (Exception ex) {
```

```
                Throwable cause = ex.getCause();

                String message =
                    cause == null
                    ? ex.getMessage()
                    : cause.getMessage();

                JOptionPane.showMessageDialog(
                    EventRegistrationGUI.this,
                    message,
                    "Registration Error",
                    JOptionPane.ERROR_MESSAGE
                );

                status.setText("Registration failed");
            }
        }

    }.execute();
}

private void clearFields() {

    studentId.setText("");
```

```
    studentName.setText("");

    department.setSelectedIndex(0);
    event.setSelectedIndex(0);

    status.setText("Fields cleared");
}

private void loadData(String selectedEvent) {

    status.setText("Loading registrations...");

    new SwingWorker<List<Registration>, Void>() {

        @Override
        protected List<Registration> doInBackground()
                throws Exception {

            return service.getRegistrations(selectedEvent);
        }

        @Override
        protected void done() {

            try {
```

```
        List<Registration> records = get();

        model.setRowCount(0);

        for (Registration r : records) {

            model.addRow(new Object[]{
                r.getStudentId(),
                r.getStudentName(),
                r.getDepartment(),
                r.getEventName()
            });
        }

        status.setText(
            "Records loaded: " + records.size()
        );

    } catch (Exception e) {

        status.setText("Unable to load records");

        JOptionPane.showMessageDialog(
            EventRegistrationGUI.this,
            "Database loading failed: " + e.getMessage()
```

```
                );
            }
        }

    }.execute();
}

private void exportData() {

    JFileChooser chooser = new JFileChooser();

    chooser.setSelectedFile(
        new File("registrations.csv")
    );

    int choice = chooser.showSaveDialog(this);

    if (choice != JFileChooser.APPROVE_OPTION) {
        return;
    }

    File file = chooser.getSelectedFile();

    try (PrintWriter writer =
            new PrintWriter(file)) {
```

```
            writer.println(
                "Student ID,Student Name,Department,Event"
            );

            for (int i = 0; i < model.getRowCount(); i++) {

                writer.println(
                    model.getValueAt(i, 0) + "," +
                    model.getValueAt(i, 1) + "," +
                    model.getValueAt(i, 2) + "," +
                    model.getValueAt(i, 3)
                );
            }

            JOptionPane.showMessageDialog(
                this, "Export completed successfully"
            );

        } catch (Exception e) {

            JOptionPane.showMessageDialog(
                this, "Export failed: " + e.getMessage()
            );
        }
```

```
    }

    public static void main(String[] args) {

        SwingUtilities.invokeLater(() ->
            new EventRegistrationGUI().setVisible(true)
        );
    }
}
```

Implementation notes:

- This sample uses MySQL Connector/J. Add the JDBC driver to the project.
- Update the database username and password in DBConnection.java.
- Run the SQL script before starting the Java application.
- The export operation exports the rows currently displayed in the table.
- The example loads all matching rows. Pagination is discussed in the alternative analysis below.
- For a production application, CSV values should be escaped correctly, and export should also run in a background worker for large tables.

6.9 Demonstration and Test Cases

The application must be tested using normal, invalid, and duplicate registrations.

Test Case	Input	Expected Output
TC01	Valid student details and Cultural Fest	Registration successful
TC02	Valid student details and Technical Fest	Registration successful

TC03	Empty student ID	Student ID required
TC04	Empty student name	Student name required
TC05	Same student and same event entered again	Duplicate registration error
TC06	Same student selects a different event	Registration successful
TC07	Filter Cultural Fest	Only Cultural Fest records displayed
TC08	Click Export	CSV file generated
TC09	Database unavailable	Database error displayed

Evidence to Attach

1. Screenshot of the registration form.
2. Screenshot of a successful registration.
3. Screenshot showing an invalid student ID.
4. Screenshot showing a duplicate registration error.
5. Screenshot of the JTable containing registration details.
6. Screenshot showing event filtering.
7. Screenshot of the exported CSV file.
8. Screenshot showing a database error, if tested.

6.10 Alternative Concept – Load All Registrations vs Pagination

The assignment asks for a comparison between loading all registrations into JTable and loading one page at a time using SQL LIMIT and OFFSET in SwingWorker.

Approach 1: Load All Registrations

In this approach, the application retrieves all 100,000 registrations from the database and loads them into the JTable.

Example:

```
SELECT *
FROM registrations;
```

Advantages:

- All records are available in one result set.
- No need for next-page or previous-page operations.
- Filtering can be performed on the loaded records.

Disadvantages:

- A large amount of memory is required.
- Database retrieval may take longer.
- Table creation and rendering may become slow.
- The GUI may become unresponsive if loading is done on the event-dispatch thread.

Approach 2: Pagination Using LIMIT and OFFSET

Pagination retrieves only a limited number of records at a time.

Example:

```
SELECT *
FROM registrations
ORDER BY registration_id
LIMIT 100 OFFSET 0;
```

For the next page:

```
SELECT *
FROM registrations
ORDER BY registration_id
LIMIT 100 OFFSET 100;
```

For the third page:

```
SELECT *
FROM registrations
ORDER BY registration_id
LIMIT 100 OFFSET 200;
```

Java Pagination Example

```
String sql =
   "SELECT * FROM registrations " +
   "ORDER BY registration_id " +
   "LIMIT ? OFFSET ?";


PreparedStatement ps = con.prepareStatement(sql);


ps.setInt(1, 100);
ps.setInt(2, pageNumber * 100);
```

The query can be executed inside SwingWorker so the GUI remains responsive while records are loaded.

Comparison

Feature	**Load All Records**	**Pagination**
Records loaded	All 100,000 records	Limited records per page
Initial loading	May take longer	Usually loads fewer records
Memory usage	High	Lower for the currently displayed page
GUI responsiveness	May be affected	Better when background loading is used
Navigation	Not required	Previous and Next required

Database requests	Usually one main retrieval	One retrieval for each page
Suitable for	Small datasets	Large datasets

Time Complexity

Let:

- N = total number of registrations.
- P = number of records per page.

Loading all records: The application retrieves and processes N records, so the record-processing work is O(N).

Pagination: Each page processes approximately P records, so the result-processing work per page is O(P). The database also performs query work based on the index, query plan, and OFFSET position. Large OFFSET values may become slower because the database may need to scan or skip earlier rows.

Space Complexity

Loading all records:

\[O(N) \]

The application stores all retrieved registrations in memory.

Pagination:

\[O(P) \]

The application stores only the records required for the current page, assuming previous pages are not retained.

Effect on Window Responsiveness

Loading 100,000 registrations directly in an event handler may freeze the window. Pagination reduces the amount of data retrieved at one time. Using SwingWorker keeps the database work outside the event-dispatch thread.

Therefore, pagination with background loading is more suitable for a large registration database.

6.11 Conclusion

The Campus Event Registration Desktop Application provides a complete design for managing student registrations. Java Swing is used to create the registration form, table, buttons, menus, and dialogs. JDBC connects the application to MySQL, while the service and DAO layers separate business logic from database operations.

Input validation prevents incorrect data, and a database unique constraint prevents duplicate registrations. SwingWorker keeps database operations from blocking the GUI. Pagination provides a more scalable way to display large numbers of registrations.

OVERALL CONCLUSION

This assignment demonstrates the design and implementation of event-driven graphical user interfaces using Java AWT and Swing.

Task 1 identifies problems in a fee-payment window and demonstrates separation of the GUI and service logic. Task 2 designs an event-driven hospital appointment desk using event sources, listener interfaces, and component hierarchies. Task 3 demonstrates the use of layout managers to create a responsive campus library search screen.

Task 4 explains mouse-event handling, repainting, and the use of the @Override annotation. Task 5 compares four event-handling styles: ActionListener in the frame, named inner classes, anonymous inner classes, and lambda expressions. Task 6 combines Swing components, JDBC, MySQL, input validation, duplicate prevention, and background processing in a campus event registration application.

Through these tasks, the assignment improves understanding of Java GUI programming, event delegation, layout management, listener classes, database integration, and software design.

MARK DISTRIBUTION

Task	Competency Assessed	Bloom's Level	Marks
Task 1	Diagnose and repair a poor event-handling design	K4 – Analyse	6
Task 2	Design an event-driven interface from a requirement	K4 – Analyse	8
Task 3	Design and implement layouts	K4 – Analyse	9
Task 4	Debug event handling using failing interactions	K4 – Analyse	7
Task 5	Compare event-handling styles under changing requirements	K5 – Evaluate	7
Task 6	Independently design and build a layered GUI– JDBC application	K5 – Evaluate	13
	TOTAL		**50**